

Linux for DevOps

Commands Reference + Bash Shell Scripting

Part 1 - Linux commands, basic to advanced (12 categories)

Part 2 - Bash scripting: variables, if, for, while, case, functions

Part 3 - Production issue scripts: high memory, high disk, CPU, service health

Part 4 - User, group and permission management scripts

Part 5 - File and variable scenarios

Part 6 - Cron jobs and scheduling

Part 7 - Production troubleshooting flow

PART 1 - Linux Commands: Basic to Advanced DevOps

1. File & Directory Management

Command	Description	Example	Use Case / Scenario
pwd	Show current directory	pwd	Verify where you are before executing commands
ls	List files/directories	ls	Check files in a directory
ls -la	List all files with details	ls -la	Find hidden files such as .ssh, .git, .env
cd	Change directory	cd /var/log	Navigate to application/system logs
mkdir	Create directory	mkdir backup	Create a backup or deployment directory
mkdir -p	Create nested directories	mkdir -p app/logs/archive	Create complete directory structure
touch	Create empty file/update timestamp	touch app.log	Create a log/config file
cp	Copy files/directories	cp app.conf backup/	Backup configuration
cp -r	Recursively copy directory	cp -r app/ backup/	Backup application files
mv	Move/rename file	mv old.conf new.conf	Rename configuration during deployment
rm	Remove file	rm temp.log	Delete temporary files
rm -r	Remove directory recursively	rm -r old_logs/	Remove obsolete directories
ln -s	Create symbolic link	ln -s /opt/app/current /opt/app/live	Point deployment symlink to current release
tree	Display directory structure	tree /opt/app	Understand application directory layout

2. File Viewing & Text Processing

Command	Description	Example	Use Case / Scenario
cat	Display file contents	cat app.conf	Quickly inspect configuration
less	View large files page by page	less /var/log/syslog	Analyze large production logs
head	Show beginning of file	head -20 app.log	Check first log entries
tail	Show end of file	tail -50 app.log	Check latest application errors
tail -f	Follow log in real time	tail -f app.log	Monitor application during deployment
wc	Count lines/words/characters	wc -l app.log	Count number of log entries
sort	Sort lines	sort users.txt	Sort data before analysis
uniq	Remove/count duplicates	sort access.log uniq -c	Find repeated log entries
cut	Extract columns	cut -d: -f1 /etc/passwd	Extract Linux usernames
tr	Translate/delete characters	echo "hello" tr 'a-z' 'A-Z'	Transform command output
grep	Search text	grep "ERROR" app.log	Find errors in logs
grep -i	Case-insensitive search	grep -i "error" app.log	Find ERROR/error/Error
grep -r	Recursive search	grep -r "database" /etc/app/	Find configuration references
sed	Search/replace/process text	sed 's/dev/prod/g' config.txt	Modify configuration values
awk	Process columns/data	awk '{print \$1,\$7}' access.log	Analyze web-server logs
diff	Compare files	diff old.conf new.conf	Find configuration changes

3. File Search

Command	Description	Example	Use Case / Scenario
find	Search files/directories	find /var/log -name "*.log"	Find log files
find -type f	Search files	find /app -type f	List application files
find -type d	Search directories	find /app -type d	Find application directories
find -mtime	Find by modification time	find /var/log -mtime +7	Find logs older than 7 days

Command	Description	Example	Use Case / Scenario
find -size	Find by file size	<code>find /var/log -size +1G</code>	Find huge log files
locate	Quickly search filenames	<code>locate nginx.conf</code>	Quickly find known files
which	Find executable location	<code>which python3</code>	Determine which binary is being used
whereis	Locate binary/man/source	<code>whereis nginx</code>	Investigate installed software

4. Permissions & Users

Command	Description	Example	Use Case / Scenario
whoami	Show current user	<code>whoami</code>	Verify execution identity
id	Show user/group IDs	<code>id</code>	Troubleshoot permissions
who	Show logged-in users	<code>who</code>	Check active sessions
groups	Show user's groups	<code>groups anil</code>	Verify group membership
passwd	Change password	<code>passwd</code>	Change user password
useradd	Create user	<code>sudo useradd devops</code>	Create service/admin user
usermod	Modify user	<code>sudo usermod -aG docker devops</code>	Add user to Docker group
userdel	Delete user	<code>sudo userdel devops</code>	Remove unused account
groupadd	Create group	<code>sudo groupadd developers</code>	Create access-control group
chmod	Change permissions	<code>chmod 755 script.sh</code>	Make deployment script executable
chown	Change ownership	<code>sudo chown app:app app.log</code>	Fix application file ownership
chgrp	Change group ownership	<code>sudo chgrp developers app.conf</code>	Give group access
umask	Set default permissions	<code>umask 027</code>	Control permissions of newly created files

5. Processes & Performance

Command	Description	Example	Use Case / Scenario
ps	Show processes	<code>ps</code>	Check processes
ps aux	Detailed process list	<code>ps aux</code>	Find CPU/memory-consuming processes
ps -ef	Full process listing	<code>ps -ef</code>	Investigate running services
top	Real-time process monitoring	<code>top</code>	Investigate high CPU
htop	Interactive process monitor	<code>htop</code>	Easier process analysis
pgrep	Find process by name	<code>pgrep nginx</code>	Find Nginx PID
pidof	Find PID of program	<code>pidof nginx</code>	Get service PID
kill	Send signal to process	<code>kill 1234</code>	Gracefully stop process
kill -9	Force kill process	<code>kill -9 1234</code>	Kill stuck process as last resort
pkill	Kill by process name	<code>pkill nginx</code>	Stop processes by name
nice	Start with priority	<code>nice -n 10 ./job.sh</code>	Run background workload with lower priority
renice	Change process priority	<code>renice 10 -p 1234</code>	Reduce priority of resource-heavy process
uptime	Show uptime/load	<code>uptime</code>	Quickly check system load
free	Show memory usage	<code>free -h</code>	Investigate memory pressure
vmstat	Show CPU/memory/IO stats	<code>vmstat 2</code>	Analyze system performance
iostat	Show disk/CPU statistics	<code>iostat -xz 2</code>	Investigate disk I/O bottleneck

6. Disk & Storage

Command	Description	Example	Use Case / Scenario
df	Show filesystem space	<code>df -h</code>	Check disk capacity
du	Show directory size	<code>du -sh /var/log</code>	Find large directories
du -ah	Show file/directory sizes	<code>du -ah /var/log</code>	Identify large files

Command	Description	Example	Use Case / Scenario
lsblk	List block devices	lsblk	Inspect attached disks
blkid	Show filesystem UUID/type	sudo blkid	Identify disk/filesystem
mount	Mount filesystem	sudo mount /dev/sdb1 /data	Attach storage
umount	Unmount filesystem	sudo umount /data	Safely detach storage
fdisk	Partition disks	sudo fdisk -l	Inspect disk partitions
du -sh	Calculate directory size	du -sh /opt/app	Check application storage usage

7. Networking

Command	Description	Example	Use Case / Scenario
ip addr	Show IP/interfaces	ip addr	Find server IP
ip -br addr	Compact IP information	ip -br addr	Quickly inspect interfaces
ip link	Show interface status	ip link	Check whether interface is UP
ip route	Show routing table	ip route	Troubleshoot gateway/routing
ip route get	Show route to destination	ip route get 8.8.8.8	Determine interface/gateway used
ip neigh	Show ARP/neighbor table	ip neigh	Troubleshoot local network connectivity
ping	Test connectivity	ping -c 4 8.8.8.8	Check network reachability
ss	Show sockets/connections	ss -tulnp	Find listening ports
curl	Test HTTP/API	curl -v https://example.com	Troubleshoot API connectivity
wget	Download/test HTTP	wget https://example.com/file	Download artifacts
nc	Test TCP/UDP connection	nc -zv server 443	Check whether port 443 is reachable
dig	DNS lookup	dig github.com	Troubleshoot DNS
nslookup	DNS lookup	nslookup github.com	Verify DNS resolution
traceroute	Trace network path	traceroute 8.8.8.8	Find routing/latency problems
tcpdump	Capture packets	sudo tcpdump -i eth0 port 443	Deep network troubleshooting
nmap	Scan network/ports	nmap -p 80,443 server	Authorized port/service discovery

8. Services & Systemd

Command	Description	Example	Use Case / Scenario
systemctl status	Check service status	systemctl status nginx	Determine why Nginx isn't running
systemctl start	Start service	sudo systemctl start nginx	Start application service
systemctl stop	Stop service	sudo systemctl stop nginx	Stop service for maintenance
systemctl restart	Restart service	sudo systemctl restart nginx	Apply configuration/change
systemctl reload	Reload configuration	sudo systemctl reload nginx	Apply config without full restart
systemctl enable	Enable at boot	sudo systemctl enable nginx	Automatically start after reboot
systemctl disable	Disable at boot	sudo systemctl disable nginx	Prevent automatic startup
journalctl	View systemd logs	journalctl -u nginx	Investigate service failures
journalctl -f	Follow logs	journalctl -u nginx -f	Monitor service in real time
systemctl list-units	List active units	systemctl list-units	Understand running services

9. Environment & Shell

Command	Description	Example	Use Case / Scenario
env	Show environment variables	env	Troubleshoot application environment
printenv	Display environment variable	printenv PATH	Check PATH configuration
export	Set environment variable	export ENV=prod	Configure application environment
echo	Print value	echo \$PATH	Verify variables
which	Locate executable	which python3	Check active binary
source	Execute file in current shell	source ~/.bashrc	Apply shell configuration
history	Show command history	history	Review previous troubleshooting commands
alias	Create command shortcut	alias ll='ls -lah'	Speed up repetitive operations
date	Show date/time	date	Verify server time
hostname	Show hostname	hostname	Identify server

10. Archives & Compression

Command	Description	Example	Use Case / Scenario
tar	Create/extract archive	tar -cvf backup.tar app/	Package application files
tar -x	Extract archive	tar -xf backup.tar	Extract deployment package
tar -czf	Create gzip archive	tar -czf backup.tar.gz app/	Compress backups
tar -xzf	Extract gzip archive	tar -xzf backup.tar.gz	Extract release artifact
gzip	Compress file	gzip app.log	Compress large log
gunzip	Decompress gzip	gunzip app.log.gz	Read compressed log

11. Processes, Jobs & Scheduling

Command	Description	Example	Use Case / Scenario
jobs	Show shell jobs	jobs	Check background jobs
bg	Resume job in background	bg %1	Continue suspended process
fg	Bring job to foreground	fg %1	Return to interactive process
nohup	Run after logout	nohup ./deploy.sh &	Run long deployment job
&	Run command in background	./backup.sh &	Continue using terminal
crontab -e	Edit scheduled jobs	crontab -e	Schedule backups/scripts
crontab -l	List cron jobs	crontab -l	Verify scheduled tasks
at	Schedule one-time job	echo "./job.sh" at 23:00	Schedule one-time maintenance

12. Git / DevOps Daily Commands

Command	Description	Example	Use Case / Scenario
git status	Show repository state	git status	Check modified/staged files
git add	Stage changes	git add app.py	Prepare file for commit
git commit	Create commit	git commit -m "Fix API"	Save change in Git history
git log	Show commit history	git log --oneline	Investigate changes
git diff	Compare changes	git diff	Review uncommitted changes
git fetch	Download remote changes	git fetch origin	Inspect remote updates safely
git merge	Combine branches	git merge origin/main	Update local branch
git pull	Fetch + merge/rebase	git pull origin main	Synchronize local repository
git push	Upload commits	git push origin main	Publish changes
git branch	List/create branches	git branch	Manage feature branches
git switch	Switch branches	git switch main	Move between branches

Command	Description	Example	Use Case / Scenario
git clone	Copy remote repository	git clone git@github.com:user/repo.git	Set up project locally
git restore	Restore file changes	git restore app.py	Discard unwanted working changes
git revert	Reverse a commit safely	git revert abc123	Roll back production change
git reset	Move branch/HEAD	git reset --soft HEAD~1	Correct local commit history

Tip: Part 1 tables continue on the following pages. All scripts below should be tested on a non-production server first.

PART 2 - Bash Scripting Basics (variables, if, for, while, case)

A1. Variables, arguments and user input

Variables have no spaces around '='. Always quote "\$var". \$1,\$2 are arguments, \$# is the argument count, \$? is the last exit code.

```
#!/bin/bash
# variables.sh - run: ./variables.sh Anil prod

NAME="DevOps"           # normal variable (no spaces around =)
readonly APP_DIR="/opt/app" # constant, cannot be changed
TODAY=$(date +%F)        # command substitution -> 2026-10-06
export ENVIRONMENT="prod" # visible to child processes

echo "Hello $NAME, today is $TODAY"
echo "Script name   : $0"
echo "First arg     : $1"
echo "Second arg    : $2"
echo "Arg count     : $# "
echo "All args      : $@"

# default value if variable is empty
ENV_NAME="${2:-dev}"
echo "Environment   : $ENV_NAME"

# read user input
read -p "Enter your service name: " SERVICE
echo "You entered: $SERVICE"

# last command status
ls /tmp > /dev/null
echo "Exit code of last command: $?" # 0 = success, non-zero = failure

# arrays
SERVERS=("web01" "web02" "db01")
echo "First server : ${SERVERS[0]}"
echo "All servers  : ${SERVERS[@]}"
echo "Total        : ${#SERVERS[@]}"
```

A2. if / elif / else conditions

Use [[]] for tests. Common file tests: -f file, -d directory, -e exists, -r readable, -w writable, -x executable, -s non-empty. Numeric: -eq -ne -gt -lt -ge -le.

```
#!/bin/bash
# if_conditions.sh

FILE="/etc/hosts"
DIR="/var/log"
NUM=75
NAME="nginx"

# 1. file / directory checks
if [[ -f "$FILE" ]]; then
    echo "$FILE exists and is a regular file"
else
    echo "$FILE not found"
fi

if [[ -d "$DIR" ]]; then
    echo "$DIR directory exists"
fi

# 2. numeric comparison with elif
if [[ $NUM -ge 90 ]]; then
    echo "CRITICAL: usage is $NUM%"
elif [[ $NUM -ge 70 ]]; then
    echo "WARNING: usage is $NUM%"
else
    echo "OK: usage is $NUM%"
fi

# 3. string comparison
if [[ "$NAME" == "nginx" ]]; then
    echo "Web server selected"
fi

# 4. AND / OR
if [[ -f "$FILE" && -r "$FILE" ]]; then
    echo "File exists AND is readable"
fi
if [[ "$NAME" == "nginx" || "$NAME" == "apache" ]]; then
    echo "Supported web server"
fi

# 5. check if a command succeeded
if systemctl is-active --quiet "$NAME"; then
    echo "$NAME is running"
else
    echo "$NAME is NOT running"
fi

# 6. check if a command exists
if command -v docker > /dev/null 2>&1; then
    echo "docker is installed"
else
    echo "docker is NOT installed"
fi

# 7. run only as root
if [[ $EUID -ne 0 ]]; then
    echo "Please run as root (use sudo)"
    exit 1
fi
```

A3. for loops

Use for loops to repeat work over a list of servers, files or numbers.

```
#!/bin/bash
# for_loops.sh

# 1. loop over a list
for SERVER in web01 web02 db01; do
    echo "Checking $SERVER ..."
done

# 2. loop over a number range
for i in {1..5}; do
    echo "Iteration $i"
done

# 3. C-style loop
for ((i=0; i<3; i++)); do
    echo "Counter = $i"
done

# 4. loop over files
for FILE in /var/log/*.log; do
    echo "$FILE -> $(wc -l < "$FILE") lines"
done

# 5. loop over an array
SERVICES=("nginx" "docker" "sshd")
for SVC in "${SERVICES[@]}"; do
    if systemctl is-active --quiet "$SVC"; then
        echo "$SVC : RUNNING"
    else
        echo "$SVC : STOPPED"
    fi
done

# 6. ping a list of hosts
for HOST in 8.8.8.8 1.1.1.1 github.com; do
    if ping -c 1 -W 2 "$HOST" > /dev/null 2>&1; then
        echo "$HOST is reachable"
    else
        echo "$HOST is NOT reachable"
    fi
done

# 7. compress every log older than 7 days
for F in $(find /var/log/app -name "*.log" -mtime +7); do
    gzip "$F" && echo "Compressed $F"
done
```

A4. while loops

while repeats as long as the condition is true. Great for counters, reading files line by line and retry logic.

```
#!/bin/bash
# while_loops.sh

# 1. simple counter
COUNT=1
while [[ $COUNT -le 5 ]]; do
    echo "Count: $COUNT"
    COUNT=$((COUNT + 1))
done

# 2. read a file line by line (safe way)
while IFS= read -r LINE; do
    echo "Server: $LINE"
done < servers.txt

# 3. wait until a port is open (retry logic with timeout)
HOST="localhost"; PORT=8080; TRIES=0; MAX=10
while ! nc -z "$HOST" "$PORT"; do
    TRIES=$((TRIES + 1))
    if [[ $TRIES -ge $MAX ]]; then
        echo "Service did not start after $MAX attempts"
        exit 1
    fi
    echo "Waiting for $HOST:$PORT (attempt $TRIES/$MAX) ..."
    sleep 3
done
echo "Port $PORT is open"

# 4. infinite monitoring loop (Ctrl+C to stop)
while true; do
    echo "$(date '+%F %T') load: $(cut -d' ' -f1 /proc/loadavg)"
    sleep 5
done

# 5. until loop - opposite of while
N=1
until [[ $N -gt 3 ]]; do
    echo "until loop: $N"
    N=$((N + 1))
done
```

A5. case statement and functions

case is a cleaner replacement for many if/elif checks. Functions make scripts reusable. 'return' sets the function exit code.

```
#!/bin/bash
# service_ctl.sh - usage: ./service_ctl.sh start|stop|status|restart

SERVICE="nginx"

log() { echo "[$(date '+%F %T')] $*"; }

check_root() {
    if [[ $EUID -ne 0 ]]; then
        log "ERROR: run as root"
        return 1
    fi
}

check_root || exit 1

case "$1" in
    start)
        log "Starting $SERVICE"
        systemctl start "$SERVICE"
        ;;
    stop)
        log "Stopping $SERVICE"
        systemctl stop "$SERVICE"
        ;;
    restart)
        log "Restarting $SERVICE"
        systemctl restart "$SERVICE"
        ;;
    status)
        systemctl status "$SERVICE" --no-pager
        ;;
    *)
        echo "Usage: $0 {start|stop|restart|status}"
        exit 1
        ;;
esac
```

PART 3 - Production Issue Scripts

B1. High memory usage monitor

Checks memory usage, lists top memory-consuming processes and writes an alert to a log. Run from cron every 5 minutes.

```
#!/bin/bash
# check_memory.sh
THRESHOLD=80                                # alert when memory used >= 80%
LOG="/var/log/memory_alert.log"

# used% = (total - available) / total * 100
TOTAL=$(free -m | awk '/^Mem:/ {print $2}')
AVAIL=$(free -m | awk '/^Mem:/ {print $7}')
USED_PCT=$(( (TOTAL - AVAIL) * 100 / TOTAL ))

echo "$(date '+%F %T') Memory used: ${USED_PCT}%"

if [[ $USED_PCT -ge $THRESHOLD ]]; then
{
    echo "====="
    echo "$(date '+%F %T') HIGH MEMORY on $(hostname): ${USED_PCT}%"
    echo "--- Top 5 memory consuming processes ---"
    ps -eo pid,user,%mem,%cpu,comm --sort=-%mem | head -6
    echo "--- free -h ---"
    free -h
} >> "$LOG"

# optional: mail alert (needs mailx configured)
# mail -s "High memory on $(hostname)" ops@example.com < "$LOG"

# optional: clear page cache (only if you understand the impact)
# sync; echo 3 > /proc/sys/vm/drop_caches
fi
```

B2. High disk usage monitor with cleanup

Loops over every mounted filesystem, alerts above a threshold, and shows the largest directories. Optionally deletes old logs.

```
#!/bin/bash
# check_disk.sh
THRESHOLD=85
LOG="/var/log/disk_alert.log"
LOG_DIR="/var/log/app"
KEEP_DAYS=7

# 1. check every filesystem
df -hP | grep -vE '^Filesystem|tmpfs|cdrom|udev' | awk '{print $5 " " " $6}' | \
while read -r USAGE MOUNT; do
    PCT=${USAGE%\%}                                # strip the % sign
    if [[ $PCT -ge $THRESHOLD ]]; then
        echo "$(date '+%F %T') ALERT: $MOUNT is ${PCT}% full on $(hostname)" | tee -a "$LOG"
        echo "Top 5 largest directories in $MOUNT:" >> "$LOG"
        du -xh "$MOUNT" 2>/dev/null | sort -rh | head -5 >> "$LOG"
    fi
done

# 2. cleanup: delete logs older than KEEP_DAYS, compress logs older than 2 days
if [[ -d "$LOG_DIR" ]]; then
    find "$LOG_DIR" -name "*.log" -mtime +2 -exec gzip -f {} \;
    find "$LOG_DIR" -name "*.gz" -mtime +"$KEEP_DAYS" -delete
    echo "$(date '+%F %T') Cleanup done in $LOG_DIR" >> "$LOG"
fi

# 3. also check inode usage (disk can be "full" with free space left)
df -i | awk 'NR>1 && $5+0 >= 90 {print "Inode alert: " $6 " " " $5}' >> "$LOG"
```

B3. High CPU / load check

Compares 1-minute load average against the number of CPU cores.

```
#!/bin/bash
# check_cpu.sh
CORES=$(nproc)
LOAD=$(cut -d' ' -f1 /proc/loadavg)
LOAD_INT=${LOAD%.*}

echo "Cores: $CORES    1-min load: $LOAD"

if [[ $LOAD_INT -ge $CORES ]]; then
    echo "$(date '+%F %T') HIGH LOAD on $(hostname): $LOAD (cores: $CORES)" >> /var/log/cpu_alert.log
    ps -eo pid,user,%cpu,%mem,comm --sort=-%cpu | head -6 >> /var/log/cpu_alert.log
fi
```

B4. Service health check with auto-restart

Checks a list of services, restarts any that are down, and records the result. Exit code makes it usable from monitoring tools.

```
#!/bin/bash
# service_monitor.sh
SERVICES=("nginx" "docker" "sshd")
LOG="/var/log/service_monitor.log"
FAILED=0

for SVC in "${SERVICES[@]}; do
    if systemctl is-active --quiet "$SVC"; then
        echo "$(date '+%F %T') $SVC is running" >> "$LOG"
    else
        echo "$(date '+%F %T') $SVC is DOWN - restarting" >> "$LOG"
        systemctl restart "$SVC"
        sleep 3
        if systemctl is-active --quiet "$SVC"; then
            echo "$(date '+%F %T') $SVC recovered" >> "$LOG"
        else
            echo "$(date '+%F %T') $SVC FAILED to restart" >> "$LOG"
            FAILED=1
        fi
    fi
done

# HTTP check
CODE=$(curl -s -o /dev/null -w "%{http_code}" http://localhost)
[[ "$CODE" != "200" ]] && echo "$(date '+%F %T') HTTP check failed: $CODE" >> "$LOG"

exit $FAILED
```

PART 4 - User, Group and Permission Management

C1. Create user, group and set permissions

Creates a group, a user with a home directory, adds the user to the group, creates a shared directory and applies ownership and permissions.

```
#!/bin/bash
# create_user.sh - usage: sudo ./create_user.sh <username> <groupname>

USERNAME="$1"
GROUPNAME="$2"
SHARED_DIR="/data/$GROUPNAME"

if [[ $EUID -ne 0 ]]; then echo "Run as root"; exit 1; fi
if [[ -z "$USERNAME" || -z "$GROUPNAME" ]]; then
    echo "Usage: $0 <username> <groupname>"; exit 1
fi

# 1. create group if it does not exist
if getent group "$GROUPNAME" > /dev/null; then
    echo "Group $GROUPNAME already exists"
else
    groupadd "$GROUPNAME" && echo "Group $GROUPNAME created"
fi

# 2. create user if it does not exist
if id "$USERNAME" > /dev/null 2>&1; then
    echo "User $USERNAME already exists"
else
    useradd -m -s /bin/bash -G "$GROUPNAME" "$USERNAME"
    echo "User $USERNAME created"
    passwd -e "$USERNAME" > /dev/null 2>&1 # force password change at first login
    echo "Set a password with: passwd $USERNAME"
fi

# 3. add existing user to group (append, never forget -a)
usermod -aG "$GROUPNAME" "$USERNAME"

# 4. shared directory with permissions
mkdir -p "$SHARED_DIR"
chown root:"$GROUPNAME" "$SHARED_DIR"
chmod 2770 "$SHARED_DIR" # rwx for owner+group, none for others, setgid keeps group

# 5. verify
id "$USERNAME"
ls -ld "$SHARED_DIR"

# 6. permission cheat sheet
#  r=4  w=2  x=1      755 = rwxr-xr-x    644 = rw-r--r--
#  750 = rwxr-x---    600 = rw-----    700 = rwx-----
#  chmod u+x file |  chmod g+w file |  chmod o-rwx file
```

C2. Delete user safely

Takes a backup of the home directory, stops the user's processes, removes the user and optionally the group.

```
#!/bin/bash
# delete_user.sh - usage: sudo ./delete_user.sh <username> [groupname]

USERNAME="$1"
GROUPNAME="$2"
BACKUP_DIR="/backup/deleted_users"

if [[ $EUID -ne 0 ]]; then echo "Run as root"; exit 1; fi
if ! id "$USERNAME" > /dev/null 2>&1; then
    echo "User $USERNAME does not exist"; exit 1
fi

read -p "Really delete user $USERNAME? (yes/no): " ANSWER
[[ "$ANSWER" != "yes" ]] && { echo "Cancelled"; exit 0; }

# 1. back up home directory
mkdir -p "$BACKUP_DIR"
tar -czf "$BACKUP_DIR/${USERNAME}_${date +%F}.tar.gz" "/home/$USERNAME" 2>/dev/null
echo "Home directory backed up to $BACKUP_DIR"

# 2. stop all running processes of the user
pkill -u "$USERNAME" 2>/dev/null
sleep 2

# 3. delete user (-r removes home directory and mail spool)
userdel -r "$USERNAME" && echo "User $USERNAME deleted"

# 4. delete group only if it is empty and given
if [[ -n "$GROUPNAME" ]] && getent group "$GROUPNAME" > /dev/null; then
    MEMBERS=$(getent group "$GROUPNAME" | cut -d: -f4)
    if [[ -z "$MEMBERS" ]]; then
        groupdel "$GROUPNAME" && echo "Group $GROUPNAME deleted"
    else
        echo "Group $GROUPNAME still has members: $MEMBERS (not deleted)"
    fi
fi

# 5. find files still owned by the deleted user (orphaned UID)
# find / -xdev -nouser 2>/dev/null
```

PART 5 - File and Variable Scenarios

D1. File handling scenarios

Common real-world file tasks: existence checks, timestamped backups, appending, here-documents, reading config, safe editing and temp files.

```
#!/bin/bash
# file_scenarios.sh
CONFIG="/etc/app/app.conf"
BACKUP_DIR="/backup/config"
TS=$(date +%Y%m%d_%H%M%S)

# 1. check before use
[[ -f "$CONFIG" ]] || { echo "Config $CONFIG missing"; exit 1; }
[[ -s "$CONFIG" ]] || { echo "Config is empty"; exit 1; }

# 2. timestamped backup before editing
mkdir -p "$BACKUP_DIR"
cp -p "$CONFIG" "$BACKUP_DIR/app.conf.$TS"

# 3. create a file with a here-document
cat > /tmp/app.env <<EOF
APP_ENV=prod
APP_PORT=8080
DB_HOST=db01
EOF

# 4. append a line (>> appends, > overwrites!)
echo "LOG_LEVEL=info" >> /tmp/app.env

# 5. load KEY=VALUE variables from a file into the shell
set -a; source /tmp/app.env; set +a
echo "App port is $APP_PORT"

# 6. replace a value in place (keeps a .bak copy)
sed -i.bak 's/APP_ENV=prod/APP_ENV=staging/' /tmp/app.env

# 7. count and filter
ERRORS=$(grep -c "ERROR" /var/log/app/app.log)
echo "Errors today: $ERRORS"
grep "ERROR" /var/log/app/app.log | awk '{print $1}' | sort | uniq -c | sort -rn | head

# 8. redirect stdout and stderr
./deploy.sh > deploy.log 2>&1          # both into one file
./deploy.sh > out.log 2> err.log        # separate files
./noisy.sh > /dev/null 2>&1             # silence everything

# 9. temp file that is cleaned up automatically
TMP=$(mktemp)
trap 'rm -f "$TMP"' EXIT
echo "working data" > "$TMP"

# 10. count files / find big files / delete old files
echo "Log files: $(find /var/log -name '*.log' | wc -l)"
find /var/log -type f -size +500M -exec ls -lh {} \;
find /tmp -type f -mtime +7 -delete

# 11. safe delete using a variable (never run rm -rf $DIR if DIR may be empty!)
DIR="/tmp/build"
[[ -n "$DIR" && -d "$DIR" ]] && rm -rf "${DIR:?}"/*
```

D2. Variable scenarios and good scripting practice

Strict mode, string operations, arithmetic, default values, command output in variables, error handling and logging.

```
#!/bin/bash
# variable_scenarios.sh
set -euo pipefail          # exit on error, undefined variable, or pipe failure

# 1. string operations
FILE="backup_2026-10-06.tar.gz"
echo "Length      : ${#FILE}"
echo "No ext      : ${FILE%.tar.gz}"      # remove suffix
echo "No prefix   : ${FILE#backup_}"      # remove prefix
echo "Upper       : ${FILE^^}"
echo "Replace     : ${FILE//_/-}"
echo "Substring   : ${FILE:7:10}"         # 2026-10-06

# 2. arithmetic
A=10; B=3
echo "Sum: $((A + B)) Mod: $((A % B)) Power: $((A ** 2))"

# 3. default / required values
PORT=${PORT:-8080}          # use 8080 if PORT not set
: "${DB_HOST:?DB_HOST must be set}"  # stop script if DB_HOST is missing

# 4. store command output
HOSTNAME_VAL=$(hostname)
IP=$(hostname -I | awk '{print $1}')
echo "$HOSTNAME_VAL has IP $IP"

# 5. environment vs shell variables
LOCAL_VAR="only this script"
export SHARED_VAR="also child processes"
bash -c 'echo "child sees: $SHARED_VAR / $LOCAL_VAR"'

# 6. associative array (key=value)
declare -A PORTS=( [http]=80 [https]=443 [ssh]=22 )
for NAME in "${!PORTS[@]}"; do
    echo "$NAME -> ${PORTS[$NAME]}"
done

# 7. logging + error handling with trap
LOGFILE="/tmp/script.log"
log() { echo "$(date '+%F %T') [$1] ${*:2}" | tee -a "$LOGFILE"; }
trap 'log ERROR "Script failed at line $LINENO"' ERR

log INFO "Script started"
cp /does/not/exist /tmp/ || log WARN "copy failed but continuing"
log INFO "Script finished"

# 8. debugging
# bash -x script.sh          trace every command
# bash -n script.sh          syntax check only
# shellcheck script.sh        static analysis (install shellcheck)
```

PART 6 - Cron Jobs

E1. Cron syntax

A cron line has 5 time fields followed by the command. Edit with 'crontab -e', list with 'crontab -l'. System-wide jobs live in /etc/crontab and /etc/cron.d/ (they have an extra user column).

```
# +----- minute          (0-59)
# | +----- hour           (0-23)
# | | +----- day of month  (1-31)
# | | | +----- month       (1-12)
# | | | | +----- day of week (0-7, 0 and 7 = Sunday)
# | | | | |
# * * * * * command

*      = every value          */5 = every 5 units      1,15 = list of values
1-5    = range of values      @reboot = run at startup  @daily = 0 0 * * *
```

E2. Cron job examples for DevOps

Always use absolute paths, redirect output to a log file, and make scripts executable (chmod +x). Cron has a minimal PATH.

```
# open the editor: crontab -e

# every minute (testing only)
* * * * * /opt/scripts/test.sh

# every 5 minutes - memory check
*/5 * * * * /opt/scripts/check_memory.sh >> /var/log/cron_memory.log 2>&1

# every 10 minutes - service health check
*/10 * * * * /opt/scripts/service_monitor.sh >> /var/log/cron_service.log 2>&1

# every 30 minutes - disk check
*/30 * * * * /opt/scripts/check_disk.sh >> /var/log/cron_disk.log 2>&1

# every hour at minute 0 - CPU check
0 * * * * /opt/scripts/check_cpu.sh

# every day at 2:00 AM - backup
0 2 * * * /opt/scripts/backup.sh >> /var/log/backup.log 2>&1

# every day at 1:30 AM - delete logs older than 7 days
30 1 * * * find /var/log/app -name "*.log" -mtime +7 -delete

# every Sunday at 3:00 AM - weekly full backup
0 3 * * 0 /opt/scripts/weekly_backup.sh

# Monday to Friday at 9:00 AM - business-day report
0 9 * * 1-5 /opt/scripts/report.sh

# 1st of every month at midnight - monthly cleanup
0 0 1 * * /opt/scripts/monthly_cleanup.sh

# every 6 hours
0 */6 * * * /opt/scripts/sync.sh

# at reboot
@reboot /opt/scripts/start_app.sh

# set variables at the top of the crontab
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin
MAILTO=ops@example.com
```

E3. Sample backup script used by cron

A complete script that cron can run: lock file to avoid overlap, timestamped archive, retention and logging.

```
#!/bin/bash
# /opt/scripts/backup.sh
set -euo pipefail

SRC="/opt/app"
DEST="/backup/app"
KEEP_DAYS=7
LOCK="/var/run/backup.lock"
LOG="/var/log/backup.log"
TS=$(date +%Y%m%d_%H%M%S)

log() { echo "$(date '+%F %T') $*" >> "$LOG"; }

# prevent two backups running at once
if [[ -e "$LOCK" ]]; then log "Backup already running, exiting"; exit 1; fi
touch "$LOCK"; trap 'rm -f "$LOCK"' EXIT

mkdir -p "$DEST"
log "Backup started"
tar -czf "$DEST/app_$TS.tar.gz" "$SRC"
log "Created $DEST/app_$TS.tar.gz ($(du -h "$DEST/app_$TS.tar.gz" | cut -f1))"

# retention: delete backups older than KEEP_DAYS
find "$DEST" -name "app_*.tar.gz" -mtime +"$KEEP_DAYS" -delete
log "Old backups removed. Backup finished"
```

E4. Install and test the scripts

Step-by-step way to deploy any script above and schedule it.

```
sudo mkdir -p /opt/scripts
sudo cp check_memory.sh /opt/scripts/
sudo chmod 750 /opt/scripts/check_memory.sh      # make executable
sudo /opt/scripts/check_memory.sh                # test manually first
bash -n /opt/scripts/check_memory.sh             # syntax check

crontab -e                                       # add the schedule
crontab -l                                       # verify
grep CRON /var/log/syslog | tail                # Ubuntu/Debian cron log
sudo journalctl -u cron --since "1 hour ago"     # systemd view
# RHEL/CentOS: /var/log/cron and journalctl -u crond
```

PART 7 - Production Troubleshooting Flow

Do not just memorize commands. Learn this sequence so you can explain how you diagnose an outage step by step.

Problem: application not reachable

1. `ip addr` -> does the server have an IP?
2. `ip route` -> is there a default gateway?
3. `ping <gateway>` -> is the network reachable?
4. `dig <domain>` -> does DNS resolve?
5. `ss -tulnp` -> is the app listening on the port?
6. `nc -zv <host> <port>` -> is the port reachable from outside?
7. `curl -v http://<host>:<port>` -> does the app respond?
8. `systemctl status <service>` -> is the service running?
9. `journalctl -u <service>` -> what do the logs say?
10. `top / free -h / df -h` -> CPU, memory or disk problem?
11. `tcpdump` -> deep packet-level check

Interview tip: always state what each command proves and what you would check next. Example: *'ss -tulnp shows nothing on 8080, so the service is not listening - I check systemctl status and journalctl.'*